

Task 3: SQL for Data Analysis

Objective: Use SQL queries to extract and analyze data from an e-commerce database.

Tool: MySQL

Dataset: Ecommerce_SQL_Database (sample dataset)

1. Create Database

```
CREATE DATABASE Ecommerce_SQL_Database;  
USE Ecommerce_SQL_Database;
```

2. Create Tables

```
CREATE TABLE Customers (  
    customer_id INT PRIMARY KEY,  
    customer_name VARCHAR(100),  
    city VARCHAR(50),  
    email VARCHAR(100)  
);  
  
CREATE TABLE Orders (  
    order_id INT PRIMARY KEY,  
    customer_id INT,  
    order_date DATE,  
    product VARCHAR(100),  
    category VARCHAR(50),  
    quantity INT,  
    price DECIMAL(10,2),  
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)  
);
```

3. Insert Sample Data

```
INSERT INTO Customers VALUES  
(1, 'Rahul', 'Bangalore', 'rahul@gmail.com'),  
(2, 'Priya', 'Mysore', 'priya@gmail.com'),  
(3, 'Arun', 'Kolar', 'arun@gmail.com'),  
(4, 'Sneha', 'Bangalore', 'sneha@gmail.com'),  
(5, 'Kiran', 'Chennai', 'kiran@gmail.com');  
  
INSERT INTO Orders VALUES  
(101, 1, '2026-09-01', 'Laptop', 'Electronics', 1, 50000),  
(102, 2, '2026-09-02', 'Headphones', 'Electronics', 2, 2000),  
(103, 1, '2026-09-05', 'Mouse', 'Electronics', 2, 800),  
(104, 3, '2026-09-07', 'Keyboard', 'Electronics', 1, 1500),  
(105, 4, '2026-09-10', 'Shoes', 'Fashion', 2, 2500),  
(106, 5, '2026-09-12', 'T-Shirt', 'Fashion', 3, 700);
```

4. SELECT and WHERE

```
SELECT * FROM Customers;  
  
SELECT *  
FROM Orders  
WHERE price > 2000;
```

5. ORDER BY

```
SELECT *  
FROM Orders  
ORDER BY price DESC;
```

6. GROUP BY and Total Revenue

```
SELECT category, SUM(quantity * price) AS total_revenue  
FROM Orders  
GROUP BY category;
```

7. INNER JOIN

```
SELECT Customers.customer_name, Orders.product, Orders.price
FROM Customers
INNER JOIN Orders
ON Customers.customer_id = Orders.customer_id;
```

8. LEFT JOIN

```
SELECT Customers.customer_name, Orders.product
FROM Customers
LEFT JOIN Orders
ON Customers.customer_id = Orders.customer_id;
```

9. Aggregate Functions: SUM and AVG

```
SELECT SUM(quantity * price) AS total_revenue
FROM Orders;
```

```
SELECT AVG(price) AS average_price
FROM Orders;
```

10. Subquery

```
SELECT customer_name
FROM Customers
WHERE customer_id IN
(
    SELECT customer_id
    FROM Orders
    WHERE quantity * price > 5000
);
```

11. Average Revenue Per User

```
SELECT customer_id,
       SUM(quantity * price) AS total_revenue
FROM Orders
GROUP BY customer_id;

SELECT AVG(total_revenue) AS average_revenue_per_user
FROM
(
    SELECT customer_id,
           SUM(quantity * price) AS total_revenue
    FROM Orders
    GROUP BY customer_id
) AS customer_revenue;
```

12. Create a View

```
CREATE VIEW Sales_View AS
SELECT Customers.customer_name,
       Orders.product,
       Orders.category,
       Orders.quantity,
       Orders.price,
       Orders.quantity * Orders.price AS revenue
FROM Customers
INNER JOIN Orders
ON Customers.customer_id = Orders.customer_id;

SELECT * FROM Sales_View;
```

13. Create an Index

```
CREATE INDEX idx_customer_id
ON Orders(customer_id);
```

14. WHERE vs HAVING Example

```
SELECT category, SUM(price) AS total_price
FROM Orders
```

```
WHERE price > 500
GROUP BY category
HAVING SUM(price) > 2000;
```

15. Handling NULL Values

```
SELECT *
FROM Customers
WHERE email IS NULL;

SELECT COALESCE(email, 'No Email') AS email
FROM Customers;
```

16. Interview: Different Types of JOINS

```
-- INNER JOIN: matching records from both tables
-- LEFT JOIN: all records from left table + matching right records
-- RIGHT JOIN: all records from right table + matching left records
-- FULL OUTER JOIN: all records from both tables (supported by PostgreSQL)
```

17. Interview: Query Optimization

```
-- Use indexes
-- Select only required columns
-- Avoid unnecessary joins
-- Use proper WHERE conditions
-- Check the execution plan using EXPLAIN
```

Interview Questions – Simple Answers

- 1. Difference between WHERE and HAVING:** WHERE filters individual rows before grouping. HAVING filters groups after GROUP BY.
- 2. Types of joins:** INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN.
- 3. Average revenue per user:** First calculate total revenue for each customer using SUM(), GROUP BY customer_id, then calculate the average of those totals.
- 4. Subquery:** A query written inside another SQL query.
- 5. Query optimization:** Use indexes, select only required columns, avoid unnecessary joins, use suitable WHERE conditions, and check the execution plan.
- 6. View:** A view is a virtual table created from a SQL query.
- 7. NULL values:** Use IS NULL, IS NOT NULL, and COALESCE() to check or replace NULL values.